

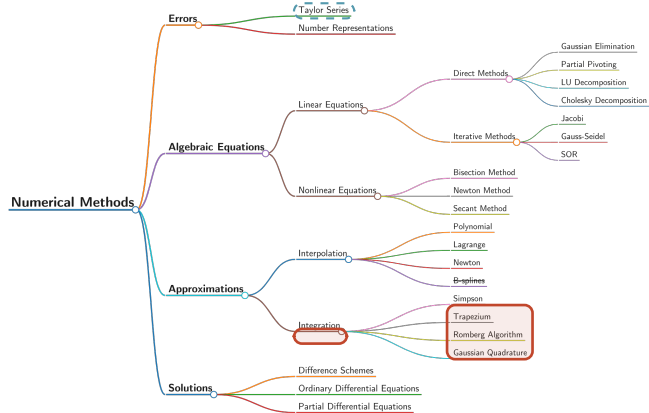
Numerical Differentiation and Integration

CTMS-MAT-13 • Difference schemes, quadrature, Romberg

Dr. David Sinden

September 10, 2026

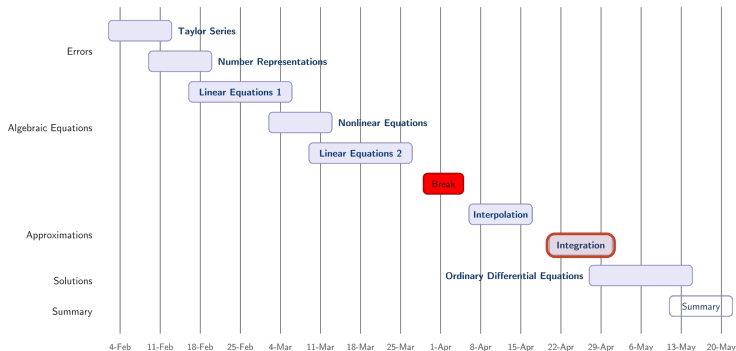
Where this sits in the course



Full outline: djps.github.io/docs/numericalmethods/intro/intro/

Schedule

Schedule 2026



Notes: djps.github.io/teaching/numerical_methods.pdf

Today

Differentiation

Richardson extrapolation

Quadrature

Romberg's algorithm

Differentiation

Forward, backward, central

The definition $\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = f'(x)$ becomes a scheme once h is small but *finite*.

Forward

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

$$\mathcal{O}(h)$$

Backward

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

$$\mathcal{O}(h)$$

Central

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

$$\mathcal{O}(h^2)$$

TAKEAWAY

Two function values, three schemes, two different orders. The *placement* of the stencil, not its size, buys the extra order.

Where the orders come from

$$\begin{aligned}f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \cdots \\f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \cdots\end{aligned}$$

Rearranging the first:

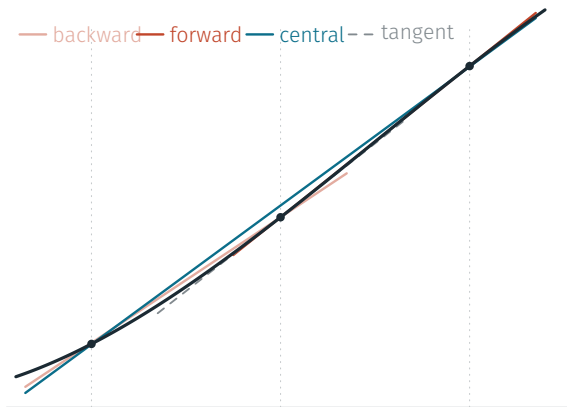
$$\frac{f(x+h) - f(x)}{h} = f'(x) + \mathcal{O}(h).$$

Subtracting the second — the f'' terms cancel:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \mathcal{O}(h^2).$$

The symmetry of the central stencil kills the leading error term. This is the whole trick, and it recurs below.

Comparing the three schemes



The second derivative: a 3-point stencil

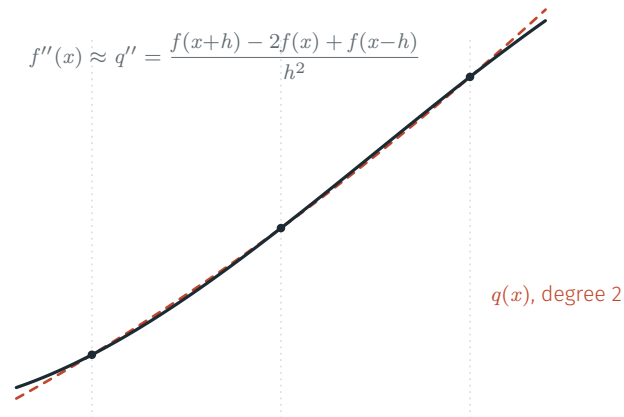
Adding the two expansions instead of subtracting them cancels f' and f''' :

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + \frac{2h^4}{4!} f^{(4)}(x) + \dots$$

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2)$$

The weights $(1, -2, 1)/h^2$ are the **3-point stencil** — a quadratic fitted through the three samples, differentiated twice.

The 3-point stencil



Richardson extrapolation

Two step sizes, one better answer

Let $\varphi(h)$ be any scheme of order k :

$$f'(x) = \varphi(h) + a_k h^k + a_{k+1} h^{k+1} + \dots$$

$f'(x)$ does not depend on h — so the *same* number is obtained with $h/2$:

$$f'(x) = \varphi(h/2) + a_k \left(\frac{h}{2}\right)^k + \dots$$

TAKEAWAY

Two evaluations at different h give two equations for one unknown. The surplus equation can be spent on eliminating the leading error term.

Eliminating the leading term

Take central differencing, $k = 2$:

$$f'(x) = \varphi(h) + ah^2 + \dots \quad (1)$$

$$f'(x) = \varphi(h/2) + \frac{ah^2}{4} + \dots \quad (2)$$

Multiply (2) by 4 and subtract (1):

$$3f'(x) = 4\varphi(h/2) - \varphi(h) + \dots \quad \Rightarrow \quad \boxed{f'(x) \approx \frac{4\varphi(h/2) - \varphi(h)}{3}}$$

- One order better than φ , for one extra evaluation;
- φ may be any scheme — the weights depend only on k ;
- the process repeats with $\varphi(h/4)$, $\varphi(h/8)$, ...

Quadrature

Integration as a sum of areas

Approximate $\int_a^b f(x) dx$ by area elements $\delta A_i = \delta x_i \cdot h_i$ — width times height.

1. Partition: $a = x_0 < x_1 < \cdots < x_n = b$, giving n subintervals.
2. Sum the rectangles:

$$\sum_{i=0}^{n-1} (x_{i+1} - x_i) f(x_i^*), \quad x_i^* \in [x_i, x_{i+1}].$$

Every rule in this lecture is one choice of the sample points x_i^* and their weights.

Riemann sums: four choices

By extremum

lower $x_i^* = \arg \min_{[x_i, x_{i+1}]} f$,
giving $L(f, P)$;

upper $x_i^* = \arg \max_{[x_i, x_{i+1}]} f$,
giving $U(f, P)$.

$$L(f, P) \leq \int_a^b f(x) dx \leq U(f, P)$$

By endpoint

$$\text{left: } \sum_{i=0}^{n-1} (x_{i+1} - x_i) f(x_i)$$

$$\text{right: } \sum_{i=0}^{n-1} (x_{i+1} - x_i) f(x_{i+1})$$

If f increases, left = lower and right = upper; if f decreases, the roles swap.

TAKEAWAY

The extremum sums *bracket* the integral — the only rules here that give a guaranteed two-sided bound.

Trapezoidal rule

Average the two endpoint heights rather than choosing one:

$$\int_{x_i}^{x_{i+1}} f(x) dx \approx (x_{i+1} - x_i) \frac{f(x_i) + f(x_{i+1})}{2}.$$

With equal spacing h , the interior points are each counted twice:

$$\int_a^b f(x) dx \approx T(f, P) = h \sum_{i=1}^{n-1} f(x_i) + \frac{h}{2} (f(x_0) + f(x_n)).$$

One function value per node, and the endpoints carry half weight.

Error of the trapezoidal rule

Theorem

Let $f \in C^2([a, b])$ and let P be an equidistant partition with spacing h . Then

$$\left| \int_a^b f(x) dx - T(f, P) \right| \leq \frac{b-a}{12} h^2 \max_{\xi \in (a,b)} |f''(\xi)|.$$

The bound comes from the remainder term in Taylor's theorem on each subinterval. More is true — the error has an **even** expansion,

$$\int_a^b f(x) dx - T(f, P) = a_2 h^2 + a_4 h^4 + \cdots,$$

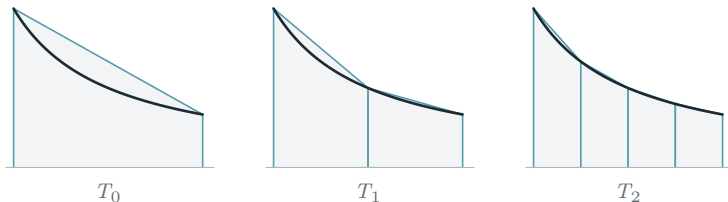
shown either by integrating by parts twice, or via linear interpolation.

Refining without recomputing

On nested partitions with $h = \frac{b-a}{2^n}$, halving h reuses every existing node:

$$T_n(f, P) = \frac{1}{2} T_{n-1}(f, P) + h \sum_{i=1}^{2^{n-1}} f(a + (2i-1)h), \quad n \geq 1.$$

Only the *new* midpoints are evaluated.



T_0, T_1, T_2 for $\int_1^3 x^{-1} dx$: each refinement keeps every node of the last.

Romberg's algorithm

Trapezoidal rule + Richardson

The trapezoidal error expansion is even in h , and nested partitions supply $\varphi(h), \varphi(h/2), \varphi(h/4), \dots$ almost for free. Richardson can therefore be applied repeatedly.

Set $R_i^0 = T_i(f, P)$, the trapezoidal value on 2^i subintervals. Eliminating the leading term at each level:

$$R_i^k = R_i^{k-1} + \frac{1}{4^k - 1} \left(R_i^{k-1} - R_{i-1}^{k-1} \right)$$

Column $k = 1$ is $\mathcal{O}(h^4)$, column $k = 2$ is $\mathcal{O}(h^6)$, and so on.

The Romberg table

nodes	$k = 0$	$k = 1$	$k = 2$	$\dots$
2^0	R_0^0			
2^1	R_1^0	R_1^1		
2^2	R_2^0	R_2^1	R_2^2	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

- column 0: trapezoidal values, each cheap given the one above;
- each column to the right: one Richardson step, two orders better;
- the answer is the bottom-right entry.

Romberg: worked example

$$\int_1^3 \frac{1}{x} dx = \ln 3 = 1.09861 \dots$$

Trapezoidal column

$$R_0^0 = 2 \cdot \frac{1}{2} \left(1 + \frac{1}{3} \right) = \frac{4}{3}$$

$$R_1^0 = \frac{1}{2} \left(\frac{5}{2} + \frac{5}{6} \right) = \frac{7}{6}$$

$$R_2^0 = \frac{67}{60}$$

Extrapolated columns

$$R_1^1 = \frac{7}{6} + \frac{1}{3} \left(\frac{7}{6} - \frac{4}{3} \right) = \frac{10}{9}$$

$$R_2^1 = \frac{67}{60} + \frac{1}{3} \left(\frac{67}{60} - \frac{7}{6} \right) = \frac{11}{10}$$

$$R_2^2 = \frac{11}{10} + \frac{1}{15} \left(\frac{11}{10} - \frac{10}{9} \right) = \frac{742}{675}$$

$R_2^2 \approx 1.09926$ against $\ln 3 \approx 1.09861$: three trapezoidal values, four correct digits.

Choosing h from the error bound

For $\int_1^3 \frac{1}{x} dx$, find h giving an error below 10^{-2} .

$f''(x) = 2/x^3$, so $\sup_{[1,3]} |f''| = 2$, and with $b - a = 2$:

$$\frac{2}{12} h^2 \cdot 2 = \frac{h^2}{3} < 10^{-2} \implies h < \frac{\sqrt{3}}{10} \approx 0.173.$$

With $h = (b - a)/2^n = 2/2^n$:

$$\frac{2}{2^n} < \frac{\sqrt{3}}{10} \implies 2^n > \frac{20}{\sqrt{3}} \approx 11.5 \implies n \geq 4.$$

TAKEAWAY

The bound is a *design* tool: it converts a required accuracy into a required number of function evaluations, before any computing is done.

Cancel the leading error term
and you have bought an order.

Summary

- Forward and backward differences are $\mathcal{O}(h)$; the symmetric central difference is $\mathcal{O}(h^2)$ for the same two samples.
- Adding the expansions instead of subtracting gives the $(1, -2, 1)$ stencil for f'' , also $\mathcal{O}(h^2)$.
- Richardson extrapolation combines $\varphi(h)$ and $\varphi(h/2)$ to cancel the leading error term and gain an order.
- Quadrature rules differ only in where they sample and how they weight; upper and lower Riemann sums bracket the answer, the trapezoidal rule averages the endpoints.
- Romberg = nested trapezoidal rules + repeated Richardson, giving $\mathcal{O}(h^4)$, $\mathcal{O}(h^6)$, ...

Take-home messages

1. **Symmetry is free accuracy.** Centre the stencil whenever the data allow it.
2. **Do not shrink h blindly.** Truncation error falls with h , but round-off in the difference quotient grows like $1/h$ — extrapolate rather than refine forever.
3. **Reuse nodes.** Nested partitions make each refinement cost only the new midpoints; this is what makes Romberg cheap.
4. **Use the error bound before computing.** It tells you how many evaluations the required tolerance costs.

Next: numerical methods for ordinary differential equations.